

MOPD Phase 4 — merge 스케줄을 유도하는 알고리즘

Qwen3-4B-OT3 · 5-domain MOPD · ramp02 스케줄의 역설계 · 2026-09-22 23:53 · fork 세션

질문. ckpt-25에 {tau, med, law}, 50에 {tau, med}, 75에 {tau}, 100 이후 없음 — 손으로 고른 이 스케줄을 측정 가능한 값에서 자동으로 유도할 수 있는가?

답. 상당 부분 가능하다. **두 가지만 보면 20개 결정이 전부 재현된다** — ① 이 도메인이 교사로부터 아직 먼가, ② 그런데 증류가 스스로 따라잡고 있지는 않은가. 두 조건이 동시에 참일 때만 merge한다.

단, 한 곳에서 깨진다. 다른 init(tau를 미리 0.8 심은 ns85323)에서는 이 규칙이 tau를 고르지 않는데, 실제로는 거기서 merge한 쪽이 전체 최고 성적을 냈다. 성능 진행률이 '교사 수준'에서 포화하기 때문이며, 포화하지 않는 좌표(유효계수 c)로 바꾸면 메울 수 있다 — 그 측정이 지금 돌고 있다.

1. 신호와 대상

표 1. 쓰는 신호 네 가지

신호	정의	이 신호가 답하는 질문	읽는 법
진행률 s	$(\text{학생} - \text{base}) \div (\text{교사} - \text{base})$	이 도메인에서 학생이 base에서 교사까지 몇 %나 왔나?	$s=0$ 학습 전과 같음 · $s=1$ 교사와 동급 · $s>1$ 교사를 넘어섬
남은 거리 gap	$1 - s$	아직 얼마나 남았나?	gap이 작으면 merge로 얻을 게 없다
따라잡는 속도 rate	최근 25스텝의 s 증가분 ÷ 남은 거리	그냥 두면 도착하나?	$0.4 =$ 남은 거리의 40%를 25스텝에 줄임 → 곧 도착 · $0 =$ 제자리 → 영원히 못 감
유효계수 c	학생 가중치가 교사 방향($\tau=\text{교사}-\text{base}$)으로 이동해 있는 배율	merge를 얼마나 이미 쌓아뒀나?	merge λ 는 이 값을 정확히 λ 만큼 올린다 · uniform init은 전부 0.2

rate를 Δs 가 아니라 남은 거리 대비로 재는 이유: $s=0.90$ 에서 $\Delta s=0.05$ 는 남은 10% 중 절반을 줄인 것이고, $s=0.10$ 에서 $\Delta s=0.05$ 는 남은 90% 중 5%에 불과하다. 같은 0.05가 전혀 다른 뜻이므로 나눠줘야 비교가 된다. c는 교사들이 완전히 직교하지 않아서 단순 투영이 아니라 5×5 Gram 행렬을 풀어 교차 성분을 뺀 값이다.

표 2. 설명하려는 대상 — 사람이 손으로 고른 ramp02 스케줄

결정 시점	고른 도메인	λ	비고
ck25	tau, med, law	0.2씩	
ck50	tau, med	0.2씩	
ck75	tau	0.2	
ck100	없음	-	실제 실행에서는 tau를 넣었으나, 결과를 보고 빼는 쪽이 맞다고 판단
ck125 이후	없음	-	

결정 4회 × 도메인 5개 = 이진 결정 20개. 어떤 규칙이든 이 20개를 맞추는지로 채점한다. fin과 if는 한 번도 선택되지 않았고 tau는 끝까지 선택됐다는 점이 핵심 단서다.

2. 관찰

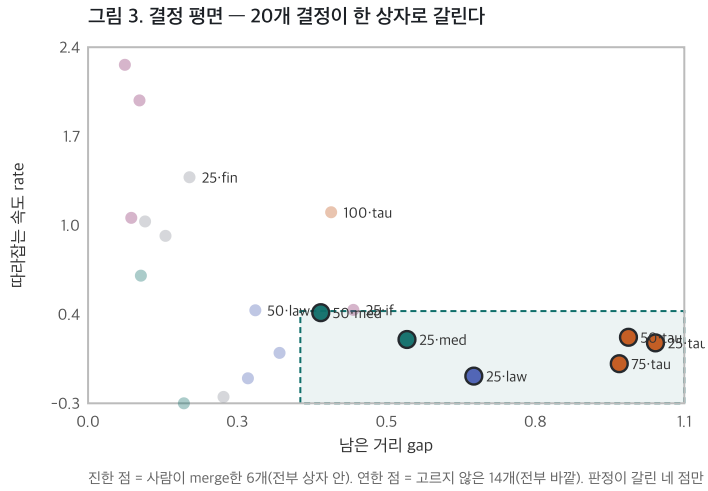
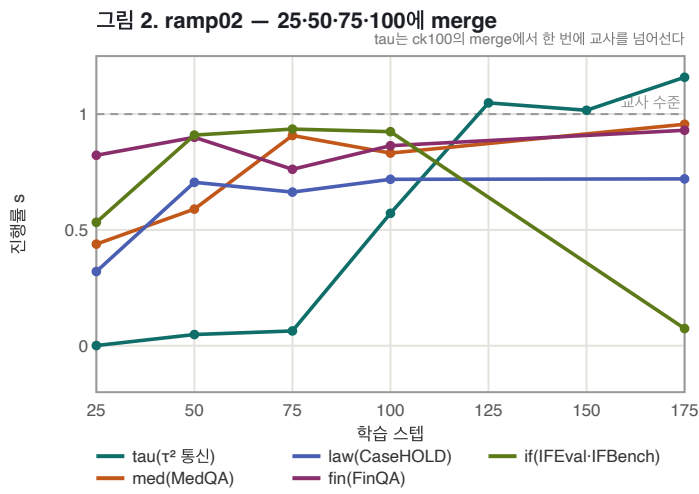
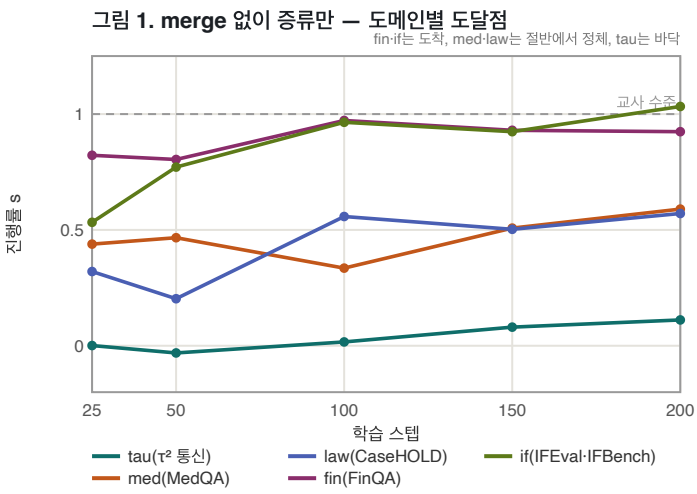


표 3. 관찰 ① — merge 없이 증류만 했을 때 도메인별 도달점 (uniform init 대조군)

ckpt	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval-IFBench)
ck25	0%	44%	32%	82%	53%
ck50	-3%	47%	20%	80%	77%
ck100	2%	34%	56%	97%	96%
ck150	8%	51%	50%	93%	92%
ck200	11%	59%	57%	92%	103%

세 가지 체질이 뚜렷하다. **fin-if**는 스스로 거의 다 따라잡는다(90% 이상). **med-law**는 절반쯤에서 멈춘다. **tau**는 200스텝을 쥐도 출발선에서 못 움직인다(11%). 사람이 고른 집합 {tau, med, law}는 정확히 '증류가 못 해주는 도메인'이고, 고르지 않은 {fin, if}는 '증류가 알아서 해주는 도메인'이다. 어느 도메인을 고르는가는 이 표만으로 설명된다.

표 4. 관찰 ② — 그런데 '진행률이 낮으면 merge'라는 단순 규칙은 성립하지 않는다 (ramp02, ✓=사람이 고름)

결정 시점	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval·IFBench)
ck25	0% ✓	44% ✓	32% ✓	82%	53%
ck50	5% ✓	59% ✓	71%	90%	91%
ck75	6% ✓	91%	66%	76%	94%
ck100	57%	83%	72%	86%	92%

ck25에서는 if를 53%인데 뻘고, ck50에서는 med를 59%인데 넣었다. 즉 '몇 % 미만이면 merge'라는 고정 임계값은 존재할 수 없다(53보다 낮아야 하는데 동시에 59보다 높아야 한다). ck100의 tau도 57%로 ck50의 med(59%)와 거의 같은데 선택이 갈린다. 수준 말고 다른 축이 하나 더 필요하다.

표 5. 관찰 ③ — 남은 거리와 따라잡는 속도를 함께 보면 20개 결정이 전부 설명된다

결정 시점	도메인	남은 거리 gap	따라잡는 속도 rate	사람의 선택	안 고른 이유
ck25	tau(τ^2 통신)	1.00	0.16	merge	
ck25	med(MedQA)	0.56	0.18	merge	
ck25	law(CaseHOLD)	0.68	-0.09	merge	
ck25	fin(FinQA)	0.18	1.41	·	가까움 (gap≤0.37)
ck25	if(IFEval·IFBench)	0.47	0.41	·	혼자 따라잡는 중 (rate≥0.40)
ck50	tau(τ^2 통신)	0.95	0.20	merge	
ck50	med(MedQA)	0.41	0.39	merge	
ck50	law(CaseHOLD)	0.29	0.41	·	가까움 (gap≤0.37)
ck50	fin(FinQA)	0.10	1.08	·	가까움 (gap≤0.37)
ck50	if(IFEval·IFBench)	0.09	2.00	·	가까움 (gap≤0.37)
ck75	tau(τ^2 통신)	0.94	0.00	merge	
ck75	med(MedQA)	0.09	0.67	·	가까움 (gap≤0.37)
ck75	law(CaseHOLD)	0.34	0.08	·	가까움 (gap≤0.37)
ck75	fin(FinQA)	0.24	-0.25	·	가까움 (gap≤0.37)
ck75	if(IFEval·IFBench)	0.06	2.27	·	가까움 (gap≤0.37)
ck100	tau(τ^2 통신)	0.43	1.15	·	혼자 따라잡는 중 (rate≥0.40)
ck100	med(MedQA)	0.17	-0.82	·	가까움 (gap≤0.37)
ck100	law(CaseHOLD)	0.28	-0.11	·	가까움 (gap≤0.37)
ck100	fin(FinQA)	0.14	0.97	·	가까움 (gap≤0.37)
ck100	if(IFEval·IFBench)	0.08	1.11	·	가까움 (gap≤0.37)

사람이 고른 6개는 모두 gap > 0.37 이면서 rate < 0.40 영역에 있고, 나머지 14개는 모두 그 바깥이다. ck100의 tau가 빠지는 이유가 특히 선명하다 — 남은 거리는 0.43으로 아직 크지만 rate가 1.15, 즉 남은 거리를 25스텝 만에 다 줄이고 있었다. 도와줄 필요가 없었던 것이다.

표 6. 관찰 ④ — tau merge는 세 번 동안 아무 효과가 없다가 네 번째에 터진다 (s_tau)

merge	직전 → 직후	겉보기 효과
ck25 → mg25	0.00 → -0.14	효과 없음
ck50 → mg50	0.05 → 0.06	효과 없음
ck75 → mg75	0.06 → 0.08	효과 없음
ck100 → mg100	0.57 → 1.00	한꺼번에 터짐

벤치마크만 보면 앞의 세 번은 낭비처럼 보인다. 그러나 낭비가 아니었다 — merge는 유효계수 **c_tau**를 **0.2씩 선형으로 쌓고** 있었고(0.2 init → 1.0), 능력은 그 누적어 어떤 선을 넘은 뒤에야 나타났다. 아래 표가 그 선형 관계를 보여준다.

표 7. 관찰 ⑤ — tau 성능은 벤치마크가 아니라 계수를 따라간다

모델	설계된 c_tau	측정된 s_tau
uniform init + 증류 200스텝	0.2	0.02 - 0.11
ns85323 init (tau를 0.8로 심음)	0.8	0.84
ramp02 mg100 (0.2 + 0.2×4)	1.0	1.00
분기 B mg100 (0.8 + 0.3)	1.1	1.06

독립적으로 만들어진 네 모델에서 **s_tau** ≈ **c_tau**가 성립한다. 도구 사용 능력은 증류로 만들어지지 않고 merge로 심는 양에 거의 비례한다. 따라서 tau에 관한 한 올바른 좌표는 '교사까지 얼마나 왔나'(s)가 아니라 '얼마나 심었나'(c)다. 이 차이가 뒤에서 알고리즘 후보를 가른다.

표 8. 관찰 ⑥ — 같은 λ가 도메인마다 전혀 다른 크기의 충격이다

도메인	교사 거리 τ	λ=0.2일 때 실제 이동 거리	if 대비
med(MedQA)	11.58	2.32	12.7×
tau(τ² 통신)	3.66	0.73	4.0×
law(CaseHOLD)	2.81	0.56	3.1×
fin(FinQA)	0.98	0.20	1.1×
if(IFEval·IFBench)	0.92	0.18	1.0×

med 교사는 if 교사보다 base에서 **12.7배** 멀다. 그래서 'λ=0.2로 통일'은 충격을 통일한 것이 전혀 아니다. 사람 스케줄을 이 단위로 보면 충격 총량이 **3.61 → 3.05 → 0.73 → 0**으로 줄어드는데, 이것이 후보 A3(예산)의 근거다.

3. 제안 알고리즘

표 9. 제안 알고리즘 — 주력 후보 3개

후보	규칙과 직관	장단점
A1. Gap-Stall Gate 벤치마크 (25스텝마다 평가 필요)	<code>merge d ⇔ 남은 거리 > 0.37</code> 그리고 따라잡는 속도 < 0.40 “아직 멀고, 그런데 스스로는 못 따라잡는 중인 도메인만 도와준다.” 두 조건 중 하나라도 어긋나면 merge하지 않는다 — 가 까우면 얻을 게 없고, 빨리 따라잡는 중이면 기다리면 된다.	장점: 20개 결정을 전부 맞힌다. 의미가 직관적이고 임계값 2개뿐. 단점: 25스텝마다 13-bench+τ² 평가가 필요해 비싸다. τ²처럼 교사를 넘어설 수 있는 도메인에서는 s가 1에서 포화해 신호가 죽는다.
A2. Coefficient Top-up 가중치만 (학습 중 무료·실시간)	<code>λ_d = clip(c*_d - c_d, 0, λ_max) · λ_d</code> 가 λ_min 이상인 도메인만 merge “각 도메인을 목표 배율 c*까지 채운다.” merge가 하는 일이 정확히 c를 λ만큼 올리는 것이므로, 결정이 곧 ‘목표까지 남은 양’이 된다. 증류가 c를 스스로 올려주면 남은 양이 줄어 자동으로 멈춘다.	장점: 평가가 전혀 필요 없고 매 스텝 켤 수 있다. tau의 ‘3번 헛일 뒤 폭발’을 누적으로 자연스럽게 설명한다. 단점: 목표 c*를 어디선가 정 해야 한다(merge-only 스케일 1회 탐색으로 구함). 지금 job 583255가 c를 재는 중.
A3. Trust-Region Budget 가중치만 (무료)	예산 <code>B_t = κ · θ_t - θ_{t-25} </code> · 부족분 큰 순서로 담되 <code>Σ λ_d τ_d ≤ B_t</code> “최근 학습이 스스로 움직인 거리보다 더 크게 흔들지 않는다.” 어느 도메인을 고를지는 다른 신호에 맡기고, 몇 개를 고를지 만 정한다. 학습이 수렴하며 느려지면 예산이 저절로 0에 수렴한다.	장점: ‘후반에는 merge를 멈춘다’를 도메인별 논리 없이 설명한다. A1·A2와 결합해서 쓰는 보조 규칙. 단점: 단독으로는 무엇을 고를지 못 정한다. κ를 정해야 한다.

A1은 ‘무엇이 부족한가’를 성능으로 보고, A2는 같은 질문을 가중치로 본다. A3는 ‘얼마나 세게 흔들어도 되는가’만 담당한다. 실전 조합은 **A2로 후보를 고르고 A3로 예산을 자르는** 형태이며, A1은 그 결과를 성능으로 검증하는 역할이다.

표 10. 나머지 후보

후보	쓰는 신호	아이디어	상태
A4 KL stall gate	도메인별 reverse KL (학생이 뽑은 문장을 교사가 얼마나 싫어하는 가)	A1과 같은 모양인데 성능 대신 KL을 쓴다. 13-bench보다 훨씬 싸고, MOPD 손실이 실제로 줄이는 양 그 자체다.	job 583258 측정 중
A5 잠재역치 모델	<code>c_d → s_d</code> 의 S자 곡선을 적합	능력이 커지는 임계 계수 <code>b_d</code> 를 추정하고, 거기까지만 빠르게 밀어넣고 멈춘다. tau의 계단식 점프를 직접 모형화한다.	583255 결과 필요
A6 한계이득/부수피해 knapsack	merge 직전·직후 쌍에서 얻은 방향미분	도메인 d를 넣었을 때 자기 성능 이득 ÷ 다른 도메인 피해로 순위를 매겨 예산까지 채운다. 이미 8쌍의 관측이 있다.	미착수
A7 Fisher 가중 merge	대각 Fisher (몇 배치의 제공 기울기)	손실이 평평한 방향으로만 밀어 피해를 줄인다. λ를 도메인마다 자동으로 다르게 만든다.	미착수
A8 순위 + 감소 예산 (탈락)	진행률 순위만	매 결정마다 최악 3·2·1·0개를 고른다. ramp02 20/20이지만 상태를 보지 않는 고정 스케줄 이라, tau가 끝까지 0.11인 대조군에서도 ck100 이후 ‘안 함’이라고 답한 다.	반례로 탈락

4. 각 알고리즘이 고르는 시나리오

표 11. 시나리오 ① — ramp02 (규칙을 만든 데이터)

결정 시점	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval·IFBench)	A1이 고르는 것	A8(탈락)이 고르는 것	사람이 고른 것
ck25	0%	44%	32%	82%	53%	{law, med, tau}	{law, med, tau}	{law, med, tau}
ck50	5%	59%	71%	90%	91%	{med, tau}	{med, tau}	{med, tau}
ck75	6%	91%	66%	76%	94%	{tau}	{tau}	{tau}
ck100	57%	83%	72%	86%	92%	{없음}	{없음}	{없음}

A1이 4개 결정을 모두 재현한다. A8도 여기서는 맞다.

표 12. 시나리오 ② — ramp03 (λ만 0.3으로 다른 별도 실행, 규칙 제작에 쓰지 않음)

결정 시점	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval·IFBench)	A1이 고르는 것	A8(탈락)이 고르는 것	사람이 고른 것
ck25	0%	44%	32%	82%	53%	{law, med, tau}	{law, med, tau}	{law, med, tau}
ck50	6%	82%	82%	94%	74%	{tau}	{if, tau}	{med, tau}
ck75	11%	89%	74%	71%	79%	{tau}	{tau}	{tau}
ck100	79%	88%	71%	67%	85%	{없음}	{없음}	{없음}

ck25는 ramp02와 같은 분기점이라 값이 동일하다. ck50에서 A1은 med를 빼자고 해서 사람과 같린다. 그런데 실제로 거기서 med를 0.3 merge한 결과 med 진행률이 0.82에서 0.72로 떨어졌다. 사람 쪽이 틀렸고 규칙이 맞았다 — 해로운 merge를 피한 것이다.

표 13. 시나리오 ③ — merge를 한 번도 하지 않은 대조군 (규칙이 끝까지 tau를 원해야 정상)

결정 시점	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval·IFBench)	A1이 고르는 것	A8(탈락)이 고르는 것
ck25	0%	44%	32%	82%	53%	{law, med, tau}	{law, med, tau}
ck50	-3%	47%	20%	80%	77%	{law, med, tau}	{law, tau}
ck100	2%	34%	56%	97%	96%	{med, tau}	{tau}
ck150	8%	51%	50%	93%	92%	{law, med, tau}	{없음}
ck200	11%	59%	57%	92%	103%	{law, med, tau}	{없음}

A1은 ck200까지 계속 {tau, med, law}를 요구한다 — 증류가 그 셋을 끝내 못 해주므로 옳다. 반면 A8은 ck100 이후 ‘안 함’이라고 답한다. 여기서 두 후보의 격이 갈린다.

표 14. 시나리오 ④ — 다른 init(ns85323, tau를 0.8로 심고 시작) 규칙이 실패하는 곳

결정 시점	tau(τ^2 통신)	med(MedQA)	law(CaseHOLD)	fin(FinQA)	if(IFEval·IFBench)	A1이 고르는 것	A8(탈락)이 고르는 것	실제로 일어난 일
ck50	56%	69%	46%	82%	87%	{law}	{law, med, tau}	분기 A가 여기서 merge → ck200에서 IF 붕괴 (§ 57%)
ck100	75%	84%	59%	88%	79%	{law}	{law, tau}	분기 B가 여기서 merge → 전체 최고 (τ^2 70.2, § 79%)
ck150	78%	95%	59%	94%	81%	{law}	{law}	
ck200	84%	82%	62%	88%	83%	{law}	{없음}	

A1은 여기서 **tau를 한 번도 고르지 않는다**. tau가 이미 75%까지 와 있어 ‘남은 거리 0.25’로 판정되기 때문이다. 그런데 분기 B는 바로 그 지점에서 tau를 merge해 τ^2 를 56.1 → 70.2로 올렸다. 원인은 명확하다 — merge는 교사를 넘어서실 수 있는데(ramp02 ck175의 s_tau는 1.16) 진행률 s는 1에서 포화해 그 여지를 보지 못한다. 계수 c는 포화하지 않으므로 A2가 이 구멍을 메울 후보다.

5. 채점과 다음 단계

표 15. 채점표

후보	ramp02 (적합)	ramp03 (OOS)	merge 없는 대조군 (OOS)	ns85323 다른 init (OOS)
A1 gap-stall	20/20	19/20 (다른 1개는 규칙이 옳음)	통과 — 끝까지 tau 요구	실패 — tau를 못 고름
A8 순위+예산 (탈락)	20/20	18/20	실패 — ck100 이후 침묵	-
A2 계수 top-up	측정 대기	측정 대기	측정 대기	이 구멍을 메울 것으로 예상 (c는 포화하지 않음)

경고: 라벨 20개에 자유 파라미터 2개이므로 ramp02 20/20 자체는 강한 증거가 아니다. 실제로 의미 있는 것은 OOS 세 칸이며, 현재 두 칸 통과·한 칸 실패다.

다음 단계

- **job 583255** — 모든 체크포인트의 유효계수 c를 측정한다. 이게 나오면 A2(계수 top-up)와 A5(잠재역치)를 같은 20-라벨 채점에 올릴 수 있고, ns85323 구멍이 메워지는지 바로 확인된다.
- **job 583258** — 도메인별 forward/reverse KL을 측정한다. A1과 같은 모양이되 25스텝마다 13-bench를 돌리지 않아도 되는 싼 대체 신호(A4)가 되는지 본다.
- **검증 실행** — ramp02 ck100에서 merge 없이 100스텝(583245), ramp03도 동일(583186). A1이 'ck100에서는 merge하지 말라'고 한 것이 실제로 맞는지 직접 확인하는 대조군이다.
- 결정되면 TaskVectorMergeCallback 에 규칙을 넣어 스케줄을 사람이 적지 않고 돌린다.

주의: 표 15의 경고를 다시 적는다. ramp02 20/20은 라벨 20개에 파라미터 2개를 맞춘 결과이므로 그 자체로는 약한 증거다. 이 보고서에서 실제로 믿을 만한 부분은 ① 표 3(종류의 도메인별 도달점이 극단적으로 다르다), ② 표 7(s_tau ≈ c_tau), ③ 표 13(대조군에서 고정 스케줄이 깨진다) 세 가지이며, 이들은 규칙 적합과 무관하게 측정된 사실이다.